

Page Replacement

2026 Semester 2 COMPSCI 340: Operating Systems

Talia Xu

Lecture 11

2.1.1

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

This Lecture Starts When a Valid Page Is Not in RAM

The TLB and page tables translate a virtual address when its page is resident in physical memory.

Today we follow the other case:

valid virtual page $\longrightarrow$ not resident $\longrightarrow$ page fault $\longrightarrow$ obtain a frame $\longrightarrow$
resume

If no frame is free, a page-replacement algorithm chooses which resident page must leave.

Demand Paging Loads a Page When It Is First Used

A process can have a valid virtual page whose contents are not currently in RAM.

- Code or file data can be loaded from its file.
- A new stack or heap page can receive a zero-filled frame.
- A page previously removed from RAM can be loaded from swap.

The PTE records that the page is absent. The first access causes a page fault so the kernel can complete the mapping.

A Page Fault Is a Controlled Detour Through the Kernel

For a valid page that is not resident:

1. The MMU finds an absent PTE and traps to the kernel.
2. The kernel checks that the virtual address is valid.
3. The kernel obtains a physical frame.
4. It loads or creates the page contents and updates the PTE.
5. The original instruction restarts and completes normally.

An invalid address cannot be completed this way; the kernel reports an invalid memory access instead.

A Free Frame Makes the Page Fault Straightforward

A free frame exists	RAM is full
Remove a frame from the free list Load the requested page Update the requested page's PTE Restart the instruction	Choose a resident victim page Write the victim back if it is dirty Reuse the victim's frame Load, update the PTE, and restart

Page replacement is needed only in the right-hand case. Its goal is to choose a victim that is unlikely to be needed soon.

The Working Set Explains Why the Choice Matters

A process's **working set** is the group of pages it is actively using during a period of time.

Example: a loop repeatedly uses its instruction page and two array pages. Its working set is roughly three pages.

If only two frames are available, one of those active pages must keep leaving. Repeated faults and disk transfers leave little time for useful work. This is **thrashing**.

Use One Reference String to Compare the Algorithms

Assume four frames and this sequence of page references:

1 2 3 4 1 2 5 1 2 3 4 5

All pages begin outside RAM. A page fault occurs whenever the referenced page is not resident.

For each algorithm, record the frame contents after each reference and count the faults. Using the same sequence makes the comparison fair.

Read a Replacement Table One Column at a Time

reference	1	2	1	3
frame 1	1	1	1	1
frame 2		2	2	2
frame 3				3
result	F	F	H	F

- A hit (H) leaves the resident set unchanged.
- A fault (F) brings the missing page into a free frame or replaces a resident page.
- The replacement rule matters only after all frames are occupied.

Optimal Replacement Is a Useful Lower Bound

Optimal replaces the page whose next use is furthest in the future.

For the comparison sequence, when page 5 arrives the frames contain 1, 2, 3, and 4. Their next uses are:

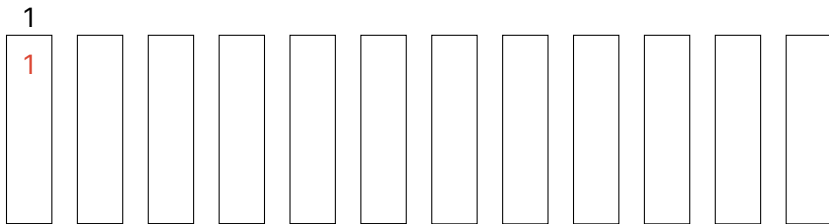
resident page	1	2	3	4
next use	next	after 1	after 1,2	after 1,2,3

Page 4 leaves because it is needed latest. A real system cannot know the future, so Optimal is a benchmark rather than an implementation.

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

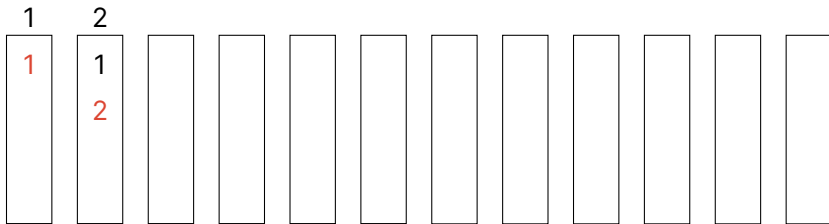
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

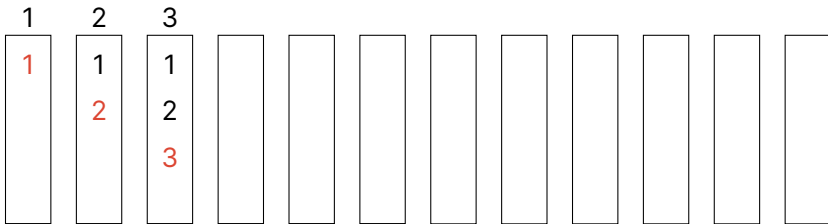
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

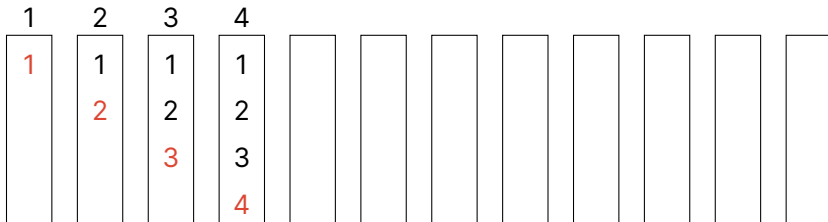
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

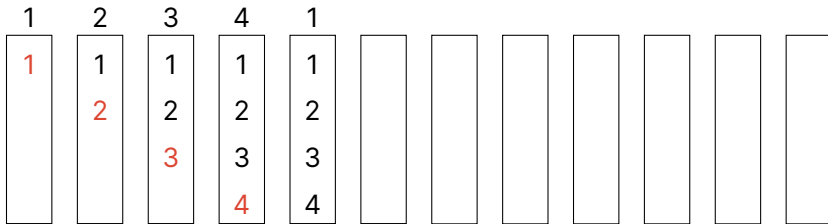
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

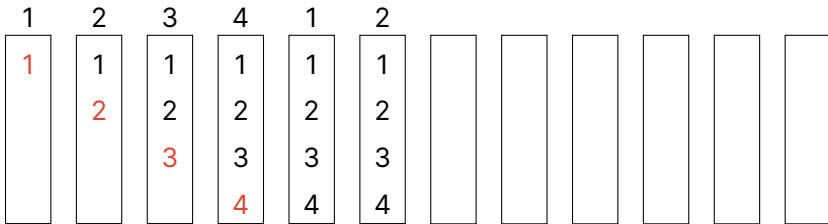
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

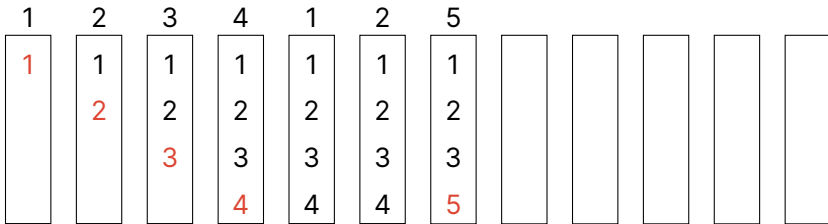
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

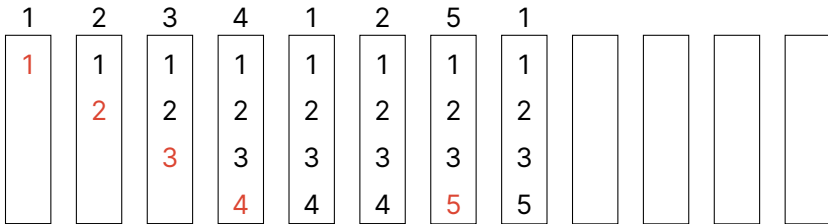
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

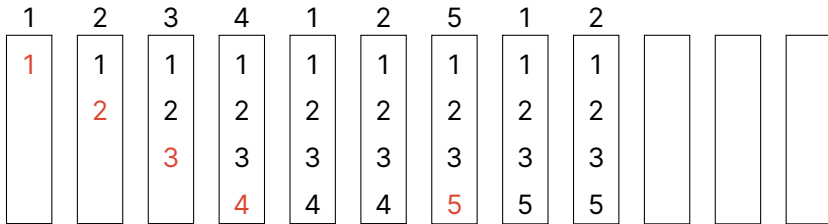
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

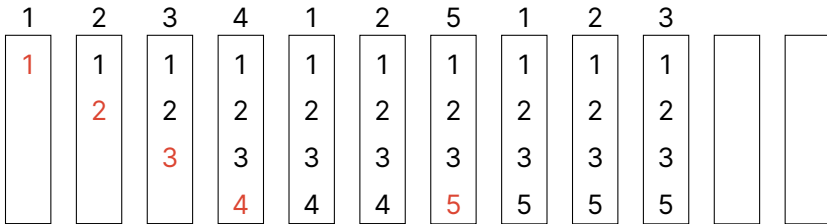
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	4	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	3	3	3	3	3	
			4	4	4	5	5	5	5	5	

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

Optimal Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	4	4
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	3	3
			4	4	4	5	5	5	5	5	5

6 page faults

FIFO Replaces the Page That Arrived First

First-in, first-out keeps pages in arrival order.

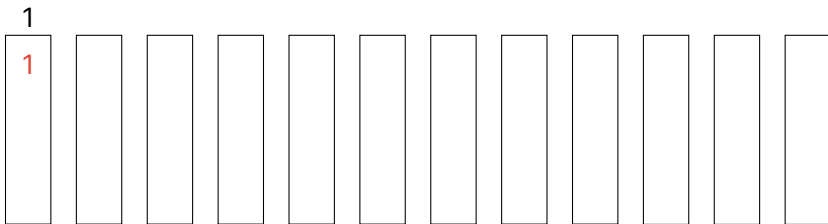
- A hit does not change that order.
- On a fault with full frames, remove the oldest resident page.
- The rule is simple, but age in RAM does not predict the next use.

FIFO needs no information about how often or how recently a page was used.

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

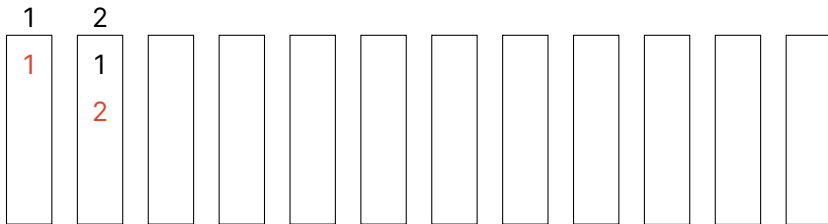
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

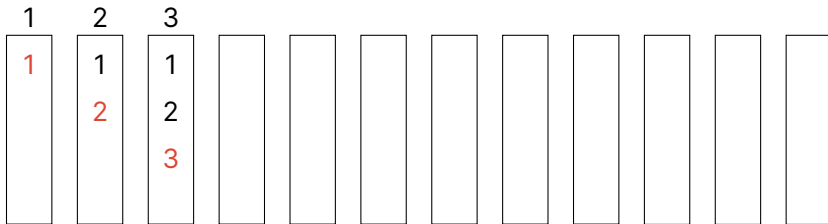
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

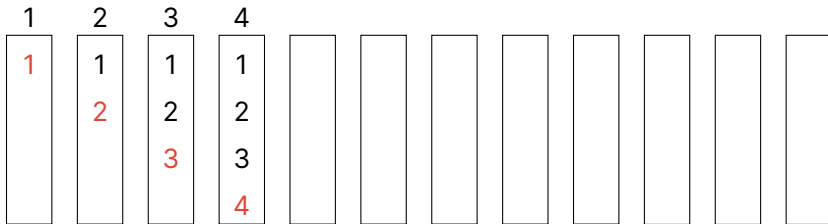
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

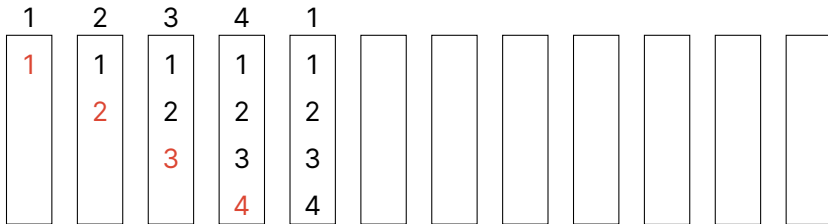
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

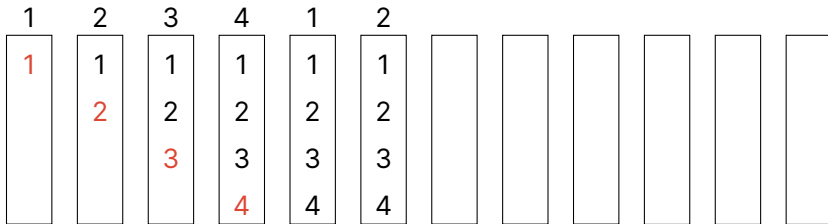
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

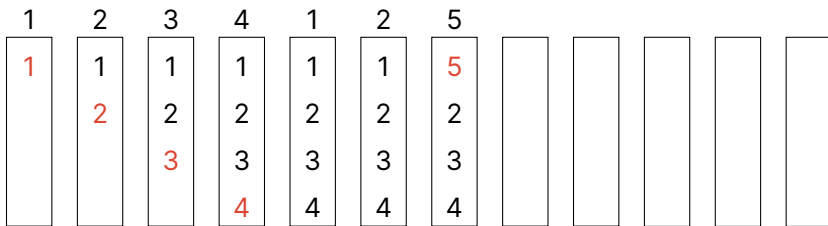
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

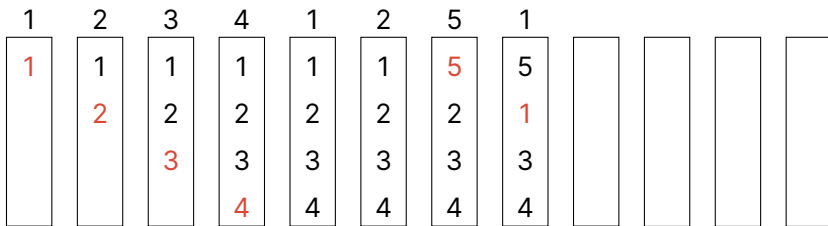
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

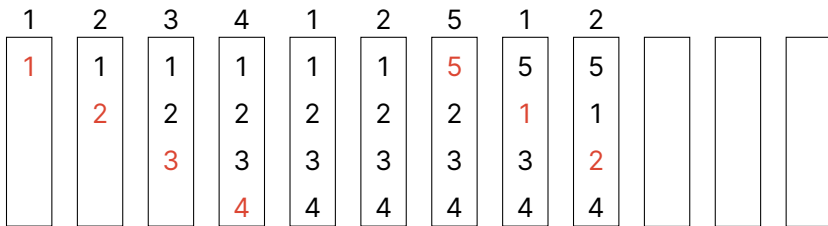
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3		
1	1	1	1	1	1	5	5	5	5		
	2	2	2	2	2	2	1	1	1		
		3	3	3	3	3	3	2	2		
			4	4	4	4	4	4	3		

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	5	5	5	5	4	
	2	2	2	2	2	2	1	1	1	1	
		3	3	3	3	3	3	2	2	2	
			4	4	4	4	4	4	3	3	

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

FIFO Example

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

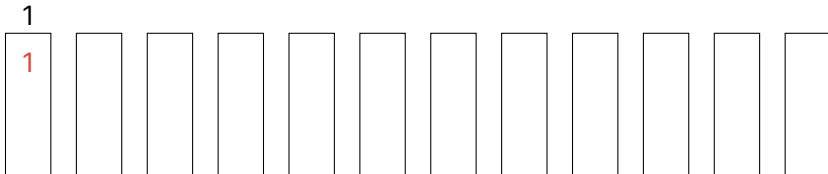
1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

10 page faults

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

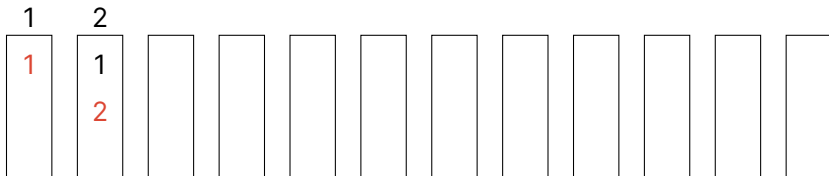
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

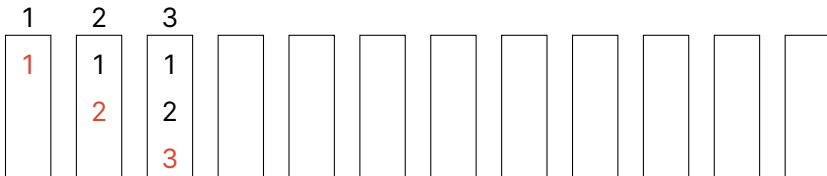
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

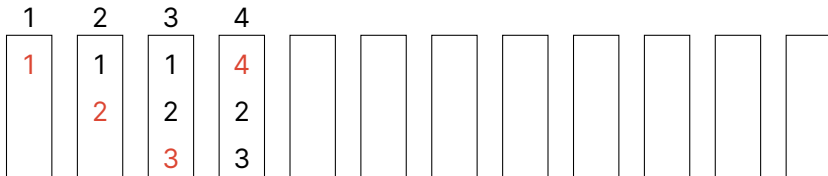
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

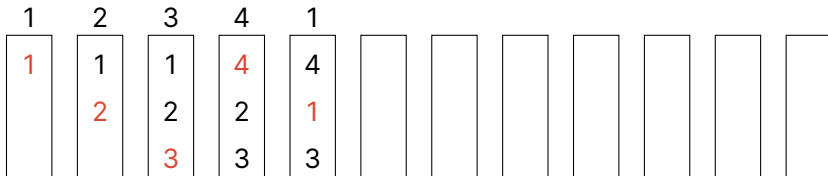
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

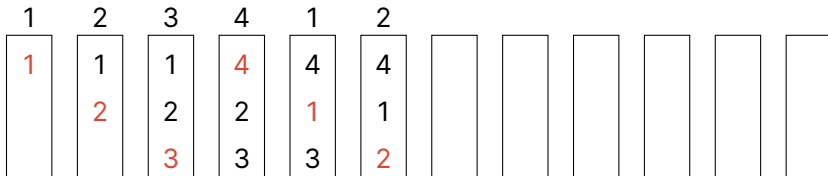
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

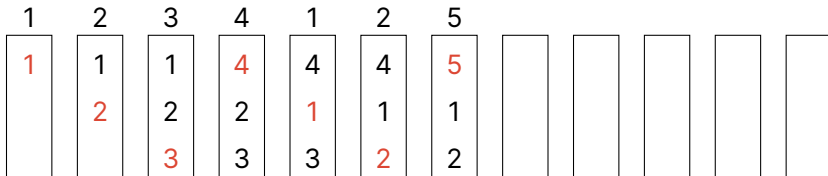
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

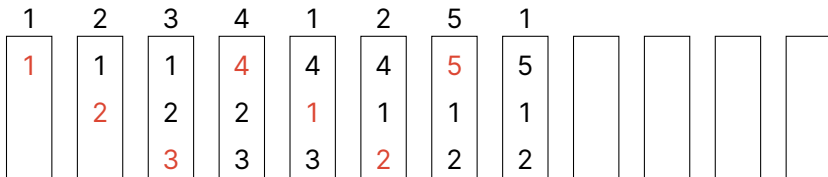
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

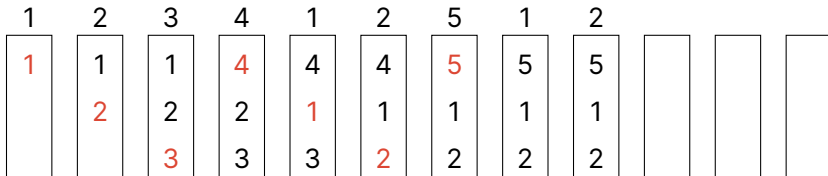
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3		
1	1	1	4	4	4	5	5	5	5		
	2	2	2	1	1	1	1	1	3		
		3	3	3	2	2	2	2	2		

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	4	4	4	5	5	5	5	5	
	2	2	2	1	1	1	1	1	3	3	
		3	3	3	2	2	2	2	2	4	

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

FIFO Example (3 Page Frames)

Assume our physical memory can only hold **3** pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

9 page faults

FIFO Can Have More Faults with More Frames

For the same reference string:

Number of frames	FIFO page faults
3	9
4	10

This is **Belady's anomaly**: giving FIFO more frames can increase the number of faults. FIFO's resident pages with four frames do not always contain the pages it would hold with three frames.

LRU Uses Recent Use as a Prediction

Least Recently Used (LRU) replaces the page that has gone the longest without being referenced.

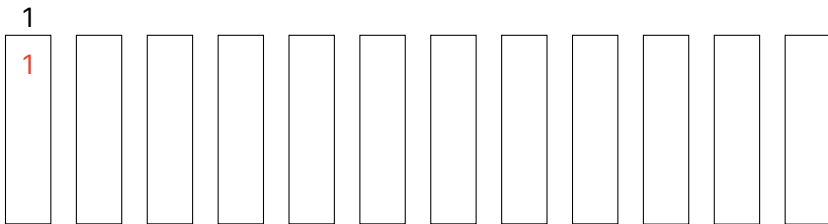
It cannot see the future, but it uses locality: a page used recently is often more likely to be used again soon.

A hit updates the page's recency. On a fault, the least recently used resident page becomes the victim.

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

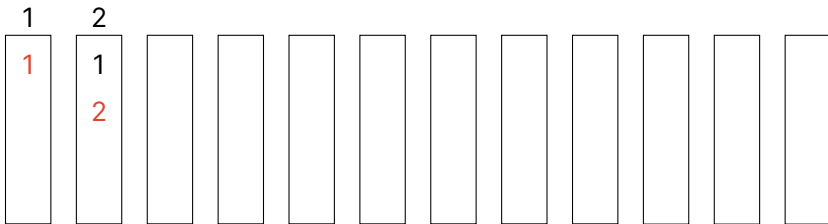
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

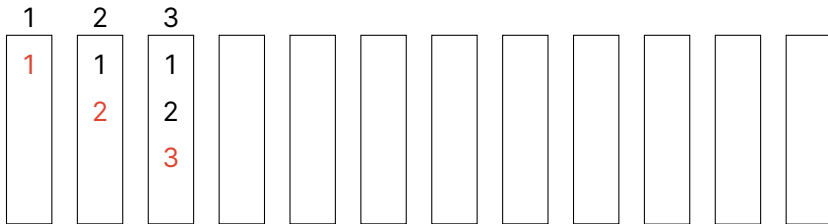
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

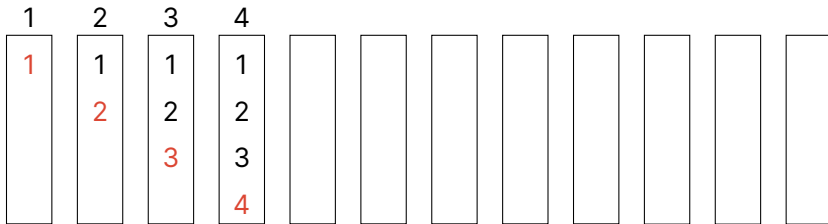
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

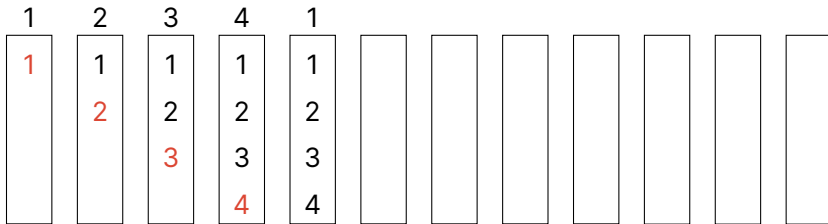
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

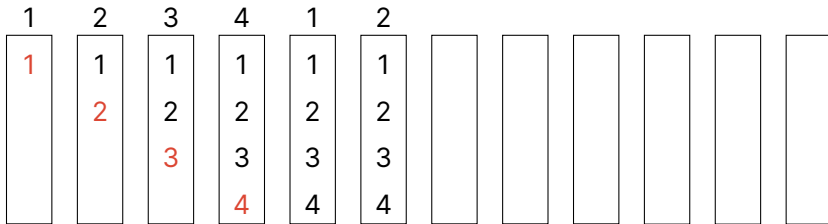
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

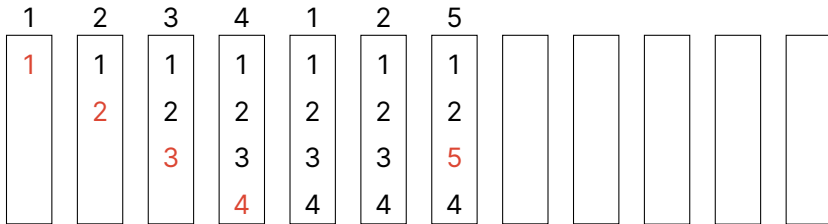
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

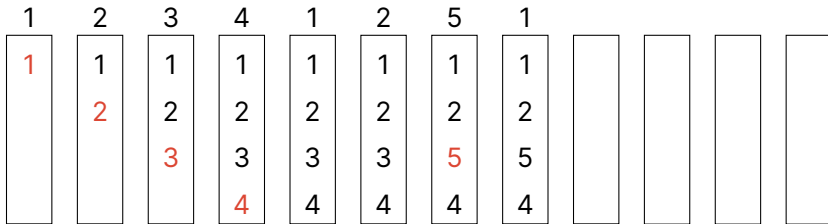
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

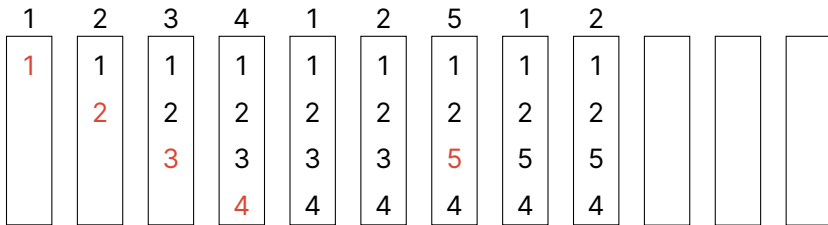
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

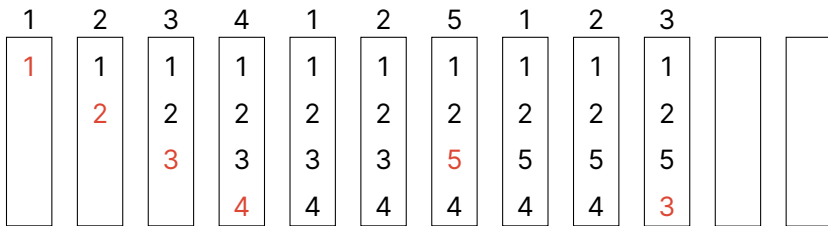
1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)



LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	
1	1	1	1	1	1	1	1	1	1	1	
	2	2	2	2	2	2	2	2	2	2	
		3	3	3	3	5	5	5	5	4	
			4	4	4	4	4	4	3	3	

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

LRU Example (Use FIFO to Break Ties)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 1 2 5 1 2 3 4 5 (all of the pages are initially on disk)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

8 page faults

Exact LRU Requires Too Much Work on Every Access

Two exact implementations illustrate the cost:

- Store a timestamp for every page, update it on every reference, and search all resident pages when choosing a victim.
- Maintain pages in recency order and move a page on every reference.

Memory accesses are far too frequent for the operating system to perform this bookkeeping in software each time. Practical systems approximate LRU using cheaper information supplied by hardware.

Clock Gives Recently Used Pages a Second Chance

Clock keeps resident pages in a circle, with one reference bit per page.

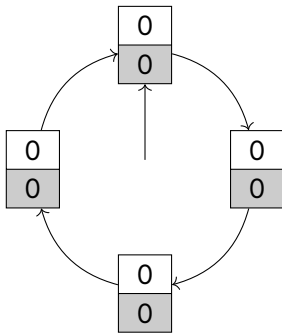
- Hardware sets a page's reference bit to 1 when it is accessed.
- On replacement, inspect the page under the clock hand.
- If its bit is 1, clear it and advance the hand.
- If its bit is 0, replace that page and advance the hand.

A recently used page survives one pass. A page that remains unused becomes eligible on a later pass.

Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

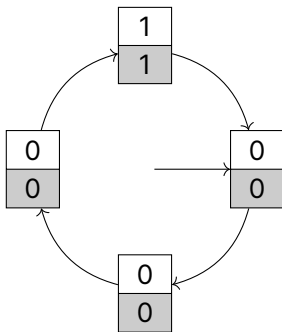


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1

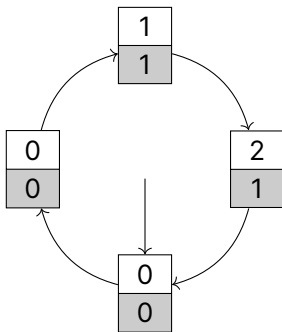


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2

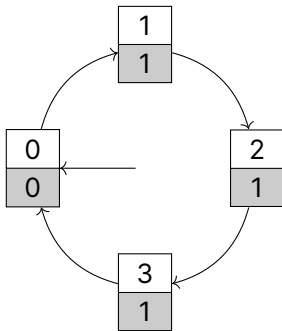


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3

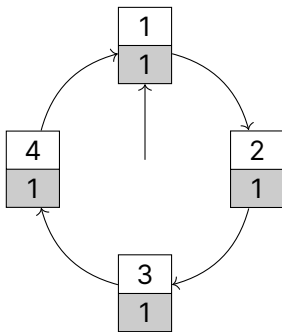


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4

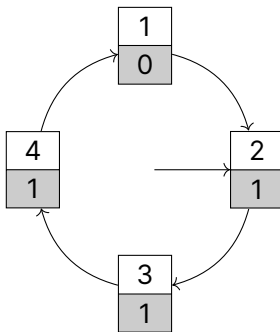


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

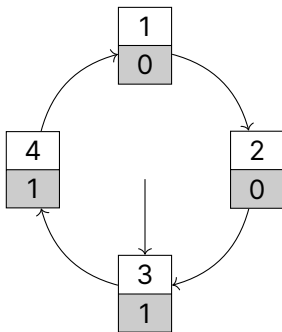


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

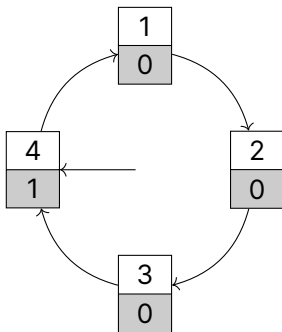


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

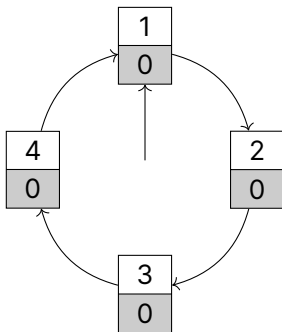


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

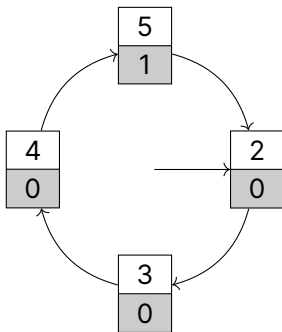


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5

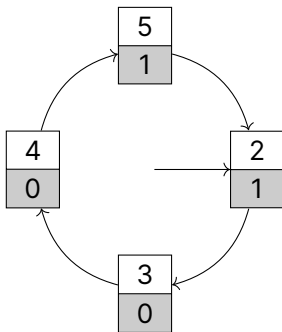


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2

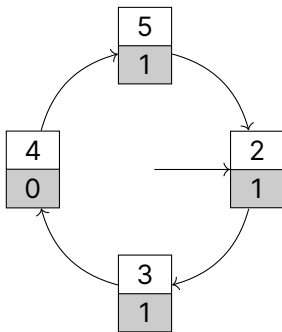


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3

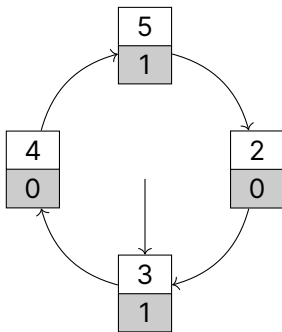


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

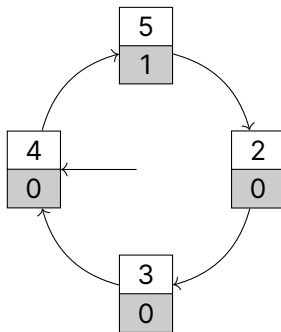


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

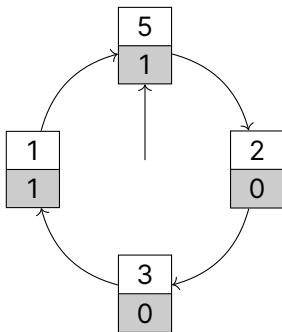


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1

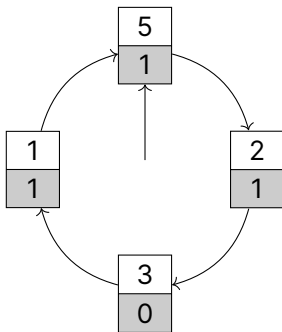


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2

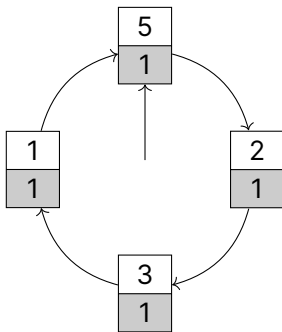


Clock Example (with Diagram)

Assume our physical memory can only hold 4 pages, and we access the following:

1 2 3 4 5 2 3 1 2 3 (all of the pages are initially on disk)

1 2 3 4 5 2 3 1 2 3



Replacement Policies Trade Information for Better Choices

Policy	Information used	Main role
Optimal	future references	lower-bound benchmark
Random	none	simple baseline
FIFO	arrival order	simple, but can behave poorly
LRU	exact recency	good idea, expensive to maintain
Clock	reference bit	practical approximation of recency

The best practical policy balances fault rate against the cost of tracking the information needed to make each decision.

Page Replacement Decides Which Resident Page Leaves

- Demand paging allows a valid virtual page to begin outside RAM.
- A page fault lets the kernel make that page resident and retry the instruction.
- When RAM is full, replacement chooses a victim frame.
- Optimal is a benchmark; FIFO is simple; LRU uses locality; Clock approximates LRU using reference bits.
- Too few frames for a working set can cause thrashing.